⚡ OhmScript DSL: The Complete Syntax Guide

Ohmic Electronic Circuit Workbench & Physical SPICE Simulation Engine

SPECIFICATION v2.0 | 34 HARDWARE COMPONENTS | PHYSICS ENGINE | CAD SPEC

⚡ OhmScript DSL: The Complete Syntax Guide & Hardware Specification

Ohmic Electronic Workbench & Physical Circuit Simulation Engine
Language Version 2.0 • Specification & Reference Manual

Table of Contents

1. Introduction to OhmScript
2. Document Structure & Anatomy
3. Top-Level Directives
4. Breadboard Coordinate System
5. Component Declarations Reference
   - Power Sources
   - Passive Components
   - Discrete Semiconductors & Actuators
   - Integrated Circuits (ICs)
   - Sensors
   - Acoustics & Actuators
   - Displays & Input Devices
   - Microcontrollers & Development Boards
6. Placement Syntax (`place`)
7. Wiring Syntax (`wire`)
8. Live Runtime Controls (`set`)
9. Instrumentation & Probing (`probe`)
10. End-to-End Verified Circuit Examples
11. Compiler Diagnostics & Troubleshooting

1. Introduction to OhmScript

OhmScript is a declarative domain-specific language (DSL) designed specifically for describing, building, and running physical electronic circuits on virtual solderless breadboards.

When compiled by the Ohmic runtime:

1. Declarations instantiate **physics-accurate non-linear SPICE/MNA mathematical models**.
2. Placements position **3D procedural Three.js component meshes** onto 830-tie-point breadboard matrices or ESD workbench mats.
3. Wires generate **3D catenary spline jumpers** with realistic electrical node routing.
4. Directives configure **dynamic operating temperatures, simulation physics, and multimeter probes**.

2. Document Structure & Anatomy

An OhmScript file (`.ohm`) is organized into five logical blocks:

```
# ── 1. Top-Level Directives ──────────────────────────
@circuit "Precision Light Sensor Switch"
@description "CdS LDR drives 2N2222 NPN transistor to switch relay"
@author "Ohmic Laboratory"
@version "2.0"
@board full_830
@temp 25
@sim on

# ── 2. Component Declarations ────────────────────────
bat    BAT1    9V
res    R1      10k       maxP=0.25W
ldr    LDR1    10k       light=0.5
npn    Q1      2N2222    hfe=150
diode  D1      1N4007
relay  RL1     5V

# ── 3. Component Placements ──────────────────────────
place BAT1 bench
place R1   E15-E19
place LDR1 D19-D20
place Q1   E25:E26:E27
place D1   F25-F26
place RL1  bench

# ── 4. Jumper Wiring ─────────────────────────────────
wire BAT1.pos  -> TOP_POS_10     red
wire BAT1.neg  -> TOP_NEG_10     black
wire TOP_POS_10 -> A15           red
wire A19       -> B26            yellow
wire TOP_NEG_10 -> A27           black
wire RL1.in    -> B25            blue

# ── 5. Runtime Instrumentation ───────────────────────
probe red   B25
probe black TOP_NEG_10
@probe_mode VOLTS_DC
set LDR1.light 0.85
```

Syntax Rules & Grammar

- **Comments**: Lines beginning with `#` or inline comments following `#` are ignored.
- **Whitespace**: Space and tabs separate tokens. Blank lines are ignored.
- **Case-Insensitivity**: All keywords (`res`, `RES`, `Res`), directives (`@temp`, `@TEMP`), and units (`k`, `K`, `uf`, `uF`) are case-insensitive.
- **Identifiers**: Component IDs (`BAT1`, `R_PULLUP`, `LED_RED`) may contain alphanumeric characters and underscores.

3. Top-Level Directives

Directives configure simulation environment variables, project metadata, and virtual test instruments.

DIRECTIVE	ARGUMENTS	DESCRIPTION	EXAMPLE
`@circuit`	`"Title"`	Name of the circuit project	`@circuit "555 Timer Blink"`
`@description`	`"Text"`	Multi-word synopsis of circuit function	`@description "Astable 1Hz square wave"`
`@author`	`"Name"`	Author or engineering team	`@author "Nikola Tesla"`
`@version`	`"SemVer"`	Schema version string	`@version "1.0.0"`
`@board`	`full_830` | `half_400`	Target breadboard layout pitch	`@board full_830`
`@temp`	Number (°C)	Workbench ambient temperature for thermal solver	`@temp 35`
`@sim`	`on` | `off`	Master simulation solver state	`@sim on`
`@probe_mode`	Mode Enum	Digital Multimeter selector switch setting	`@probe_mode VOLTS_DC`

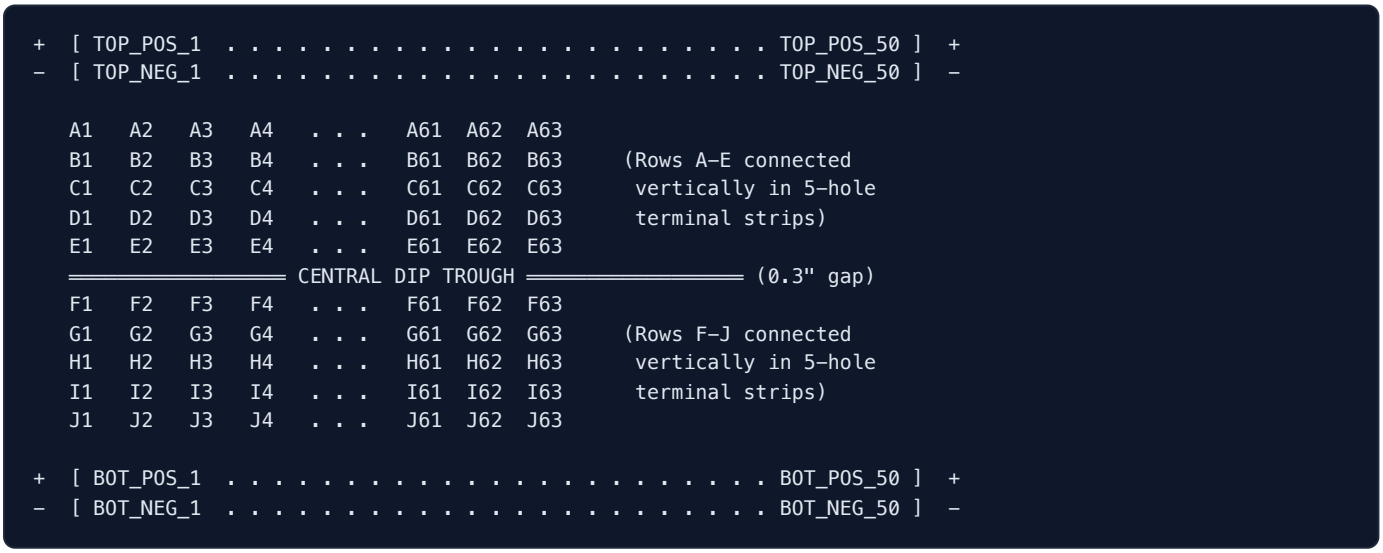
Recognized `@probe_mode` Settings:

- `VOLTS_DC` — DC Voltage Potential Difference (V)
- `VOLTS_AC` — AC RMS Voltage Potential (V_{rms})
- `CURRENT_DC` — Inline Circuit Current (mA)
- `RESISTANCE` — Unpowered Ohmmeter Resistance (Ω)
- `CONTINUITY` — Continuity Beeper ($< 30\Omega$ threshold)
- `FREQUENCY` — Signal Frequency Counter (Hz)

---

4. Breadboard Coordinate System

The virtual 830-tie-point solderless breadboard adheres to standard 2.54 mm (0.1 inch) imperial pitch geometry.



Hole Identification Rules:

1. **Terminal Columns**: Letters `A` through `J`, followed by column numbers `1` to `63`.
   - Upper section: `A1` to `E63`. Within any column, holes A, B, C, D, E are electrically shorted together.
   - Lower section: `F1` to `J63`. Within any column, holes F, G, H, I, J are electrically shorted together.
   - The Central Trough electrically isolates Row E from Row F, allowing standard 0.3" DIP ICs to straddle both sides.

2. **Power Rails**:
   - `TOP_POS_1` .. `TOP_POS_50` — Top Red (+) Power Distribution Bus.
   - `TOP_NEG_1` .. `TOP_NEG_50` — Top Blue/Black (-) Ground Bus.
   - `BOT_POS_1` .. `BOT_POS_50` — Bottom Red (+) Power Distribution Bus.
   - `BOT_NEG_1` .. `BOT_NEG_50` — Bottom Blue/Black (-) Ground Bus.
3. **Bench Mat Virtual Terminals**:
   - Items placed on the ESD bench mat (outside breadboard) expose virtual snap posts using `<id>.<terminal>` syntax:
     - Batteries: `BAT1.pos`, `BAT1.neg`
     - Speaker: `SPK1.pos`, `SPK1.neg`
     - Motor: `M1.pos`, `M1.neg`
     - Servo: `SRV1.sig`, `SRV1.vcc`, `SRV1.gnd`
     - Relay: `RL1.vcc`, `RL1.gnd`, `RL1.in`, `RL1.no`, `RL1.com`, `RL1.nc`
     - Arduino Uno: `UNO1.5v`, `UNO1.3v3`, `UNO1.gnd`, `UNO1.vin`

---

5. Component Declarations Reference

Every component is defined using the canonical grammar:

```
<type_keyword>  <unique_id>  <primary_value>  [key=value ...]
```

5.1 Power Sources

`bat` — DC Voltage Source / Battery Pack

- **Syntax**: `bat <id> <voltage> [type=alkaline|4aa]`
- **Recognized Values**: `9V`, `5V`, `6V`, `3.3V`, `12V`
- **Pins**: `pos` (+), `neg` (-)
- **Examples**:

```
bat BAT1 9V
bat BATT_PACK 6V type=4aa
bat PSU1 5V
```

`mb102` — MB102 Dual-Rail Breadboard Power Supply Module

- **Syntax**: `mb102 <id> <voltage>`
- **Pins**: Plugs directly into breadboard rails: `TOP_POS_1`, `TOP_NEG_1`, `BOT_POS_1`, `BOT_NEG_1`
- **Example**:

```
mb102 MB1 5.0V
place MB1 bench
```

5.2 Passive Components

`res` — Resistor (1/4W Metal Film)

- **Syntax**: `res <id> <resistance> [maxP=<watts>]`
- **Unit Multipliers**: `Ω`, `k`, `K`, `M` (e.g. `100`, `220`, `330`, `470`, `1k`, `4.7k`, `10k`, `47k`, `100k`, `1M`)
- **Pins**: `p1`, `p2` (non-polarized)
- **Examples**:

```
res R1 470
res R2 10k maxP=0.25W
res R3 1M
```

`pot` — Rotary Potentiometer

- **Syntax**: `pot <id> <total_resistance> [wiper=<ratio>]`
- **Wiper Range**: `0.0` (fully CCW) to `1.0` (fully CW). Default is `0.5`
- **Pins**: `t1` (Terminal 1), `w` (Wiper), `t2` (Terminal 2)
- **Example**:

```
pot POT1 10k wiper=0.75
```

`cap` — Capacitors (Ceramic, Electrolytic, Film)

- **Syntax**: `cap <id> <capacitance> [electrolytic|ceramic|film]`
- **Sub-Units**: `pF`, `nF`, `uF`, `µF`
- **Pins**:
  - Ceramic / Film: `cp1`, `cp2` (non-polarized)
  - Electrolytic: `cp1` (+), `cp2` (-) (polarized)
- **Examples**:

```
cap C1 100nF ceramic
cap C2 100uF electrolytic
cap C3 0.1uF film
```

`ind` — Inductor (Toroid Choke)

- **Syntax**: `ind <id> <inductance>`
- **Sub-Units**: `mH`, `H`
- **Pins**: `l1`, `l2` (non-polarized)
- **Example**:

```
ind L1 10mH
```

`sw` — Mechanical Switches

- **Syntax**: `sw <id> <tactile|toggle|slide>`
- **Pins**:
  - Tactile / Toggle: `sw1`, `sw2`
  - SPDT Slide: `sw1`, `com`, `sw2`
- **Examples**:

```
sw SW1 tactile
sw SW2 toggle
sw SW3 slide
```

5.3 Discrete Semiconductors & Actuators

`led` — Single-Color 5mm LED

- **Syntax**: `led <id> <color> [Vf=<voltage>]`
- **Colors**: `red` (1.9V), `green` (2.2V), `blue` (3.2V), `yellow` (2.1V), `white` (3.2V)

- **Pins**: `anode` (+), `cathode` (-)
- **Example**:

```
led LED1 red Vf=1.9V
```

`led` — 5mm Common-Cathode RGB LED

- **Syntax**: `led <id> rgb`
- **Pins**: `r` (Red Anode), `c` (Common Cathode GND), `g` (Green Anode), `b` (Blue Anode)
- **Example**:

```
led RGB1 rgb
```

`diode` — Rectifier & Signal Diodes

- **Syntax**: `diode <id> <1N4148|1N4007>`
- **Pins**: `a` (Anode +), `k` (Cathode -)
- **Examples**:

```
diode D1 1N4148    # High-speed signal (0.72V Vf)
diode D2 1N4007    # 1A 1000V Power Rectifier (0.70V Vf)
```

`npn` / `pnp` — BJT Transistors

- **Syntax**: `npn <id> <BC547|2N2222> [hfe=<gain>]`
- **Syntax**: `pnp <id> <BC557> [hfe=<gain>]`
- **Pins**:
  - BC547 / BC557: `c` (Collector), `b` (Base), `e` (Emitter)
  - 2N2222: `e` (Emitter), `b` (Base), `c` (Collector)
- **Examples**:

```
npn Q1 BC547 hfe=200
npn Q2 2N2222 hfe=150
pnp Q3 BC557 hfe=220
```

`mosfet` — Logic-Level N-Channel MOSFET

- **Syntax**: `mosfet <id> <model> [vth=<threshold>] [rdson=<ohms>]`
- **Pins**: `g` (Gate), `d` (Drain), `s` (Source)
- **Example**:

```
mosfet M1 IRLZ44N vth=2.0V rdson=0.05
```

`ldr` — CdS Photoresistor

- **Syntax**: `ldr <id> <dark_resistance> [light=<lux_ratio>]`
- **Pins**: `p1`, `p2`
- **Example**:

```
ldr LDR1 10k light=0.75
```

5.4 Integrated Circuits (ICs)

`ic555` — NE555 Precision Timer (DIP-8)

- **Syntax**: `ic555 <id>`
- **Pinout (DIP-8)**:
  - `p1`: GND (Ground)
  - `p2`: TRIG (Trigger)
  - `p3`: OUT (Output)
  - `p4`: RESET (Active Low Reset)
  - `p5`: CTRL (Control Voltage)
  - `p6`: THRES (Threshold)
  - `p7`: DISCH (Discharge)
  - `p8`: VCC (Power Supply 4.5V-16V)
- **Example**:

```
ic555 U1
place U1 E15
```

`lm358` — LM358 Dual Operational Amplifier (DIP-8)

- **Syntax**: `lm358 <id>`
- **Pinout (DIP-8)**:
  - `p1`: OUT1, `p2`: IN1-, `p3`: IN1+, `p4`: GND
  - `p5`: IN2+, `p6`: IN2-, `p7`: OUT2, `p8`: VCC
- **Example**:

```
lm358 OPAMP1
place OPAMP1 E25
```

`ic` — Generic & Specialized DIP ICs

- **Syntax**: `ic <id> <part_number>`
- **Supported Parts**:
  - `cd4017`: 16-Pin Decade Counter with 10 decoded outputs (`p1` .. `p16`)
  - `74hc595`: 16-Pin 8-Bit Shift Register with Output Latch (`p1` .. `p16`)
  - `lm393`: 8-Pin Dual Precision Voltage Comparator (`p1` .. `p8`)
  - `l293d`: 16-Pin Quad Half-H Motor Driver (`p1` .. `p16`)
  - `tb6612`: TB6612FNG Dual MOSFET Motor Driver Module (`p1` .. `p16`)
- **Examples**:

```
ic U1 cd4017
ic U2 74hc595
ic U3 lm393
ic U4 l293d
```

5.5 Sensors

All sensors are defined with the `sensor` keyword:

```
sensor <id> <type_name> [options...]
```

TYPE NAME	SENSOR MODEL	PINS	OUTPUT FORMAT	EXAMPLE
`hcsr04`	Ultrasonic Sonar	`vcc`, `trig`, `echo`, `gnd`	2cm to 400cm pulse	`sensor US1 hcsr04`
`dht22`	Temp & Humidity	`vcc`, `data`, `gnd`	1-Wire Digital (°C / %RH)	`sensor TH1 dht22`
`pir`	HC-SR501 Motion	`vcc`, `out`, `gnd`	3.3V TTL High on Motion	`sensor PIR1 pir`
`sound`	Microphone Sound	`vcc`, `out`, `gnd`	Analog Audio Envelope	`sensor MIC1 sound`
`soil`	Capacitive Moisture	`vcc`, `sig`, `gnd`	Analog Dielectric Level	`sensor SOIL1 soil`
`hall`	A3144 Hall Effect	`vcc`, `gnd`, `out`	Open-Collector Active Low	`sensor HALL1 hall`
`ir`	VS1838B 38kHz IR	`out`, `gnd`, `vcc`	38kHz NEC Demodulated	`sensor IR1 ir`

5.6 Acoustics & Actuators

`buzzer` — Active & Passive Buzzers

- **Syntax**: `buzzer <id> active|passive`
- **Pins**: `pos` (+), `neg` (-)
- **Examples**:

```
buzzer BZ1 active      # 5V fixed 2400Hz tone
buzzer BZ2 passive     # Wideband audio sounder
```

`servo` — SG90 Micro Servo (9g)

- **Syntax**: `servo <id> [angle=<0–180>]`
- **Pins**: `gnd` (Brown), `vcc` (Red 5V), `sig` (Orange PWM)
- **Example**:

```
servo SRV1 angle=90
place SRV1 bench
```

`motor` — TT DC Gearbox Motor + Wheel

- **Syntax**: `motor <id> [rpm=<speed>]`
- **Pins**: `pos` (+), `neg` (-)
- **Example**:

```
motor M1 200
place M1 bench
```

`relay` — 5V 1-Channel Optocoupled Relay

- **Syntax**: `relay <id> 5V`
- **Pins**: `vcc`, `gnd`, `in`, `no`, `com`, `nc`
- **Example**:

```
relay RL1 5V
place RL1 bench
```

5.7 Displays & Input Devices

`disp` — I2C OLED & Character LCD

- **Syntax**: `disp <id> oled|lcd`
- **Pins (I2C 4-Pin)**: `gnd`, `vcc`, `scl`, `sda`
- **Examples**:

```
disp OLED1 oled    # 0.96" SSD1306 128x64 OLED
disp LCD1  lcd     # 16x2 Character LCD with backpack
```

`keypad` — 4×4 Matrix Keypad

- **Syntax**: `keypad <id> 4x4`
- **Pins**: `r1`, `r2`, `r3`, `r4`, `c1`, `c2`, `c3`, `c4`
- **Example**:

```
keypad KP1 4x4
place KP1 bench
```

`encoder` — KY-040 Rotary Encoder

- **Syntax**: `encoder <id> ky040`
- **Pins**: `clk`, `dt`, `sw`, `vcc`, `gnd`
- **Example**:

```
encoder ENC1 ky040
place ENC1 E40:E41:E42:E43:E44
```

5.8 Microcontrollers & Development Boards

- **Syntax**: `mcu <id> nano|esp32|uno`
- **Form Factors**:
  - `nano`: Arduino Nano ATmega328P (DIP-30 straddling columns across central trough)
  - `esp32`: ESP32 NodeMCU DevKit V1 (DIP-30 straddling columns)
  - `uno`: Arduino Uno R3 (Placed on bench mat with power terminals `5v`, `3v3`, `gnd`, `vin`)
- **Examples**:

```
mcu MCU1 nano
place MCU1 E10

mcu DEV1 esp32
place DEV1 E30

mcu UNO1 uno
place UNO1 bench
```

---

6. Placement Syntax (`place`)

The `place` command anchors components to the physical breadboard grid or workbench mat:

1. Two-Hole Span (Resistors, Diodes, Capacitors, LEDs)

```
place R1   E15–E19    # Row E Column 15 to Column 19
place LED1 D19–D20    # Adjacent row pins
```

2. Multi-Pin Explicit List (Potentiometers, Transistors, Modules)

```
place POT1 E10:E11:E12        # 3-pin potentiometer
place Q1   E25:E26:E27        # 3-pin transistor C-B-E
place ENC1 E40:E41:E42:E43:E44  # 5-pin rotary encoder
```

3. Anchor Hole DIP Auto-Expansion (ICs, Dev Boards, Sockets)

Straddles the central trough automatically across rows E and F:

```
place U1   E15        # DIP-8: occupies E15–E18 and F15–F18
place U2   E30        # DIP-16: occupies E30–E37 and F30–F37
place MCU1 E10        # DIP-30 Nano: occupies E10–E24 and F10–F24
```

4. Bench Mat Outside Breadboard (`bench`)

Positions components with exposed snap terminals onto the ESD mat:

```
place BAT1  bench
place SPK1  bench
place MTR1  bench
place SRV1  bench
place RL1   bench
place UNO1  bench
place KP1   bench
```

---

7. Wiring Syntax (`wire`)

The `wire` command routs 3D flexible catenary jumpers between two electrical nodes:

```
wire  <source_endpoint>  ->  <destination_endpoint>  [color]
```

Recognized Wire Colors:

- `red` (`#ef4444`) — Positive Power (+5V / +9V)
- `black` (`#18181b`) — Ground / Negative (0V)
- `blue` (`#2563eb`) — Analog Signals & Low-Side Returns
- `green` (`#22c55e`) — Clock / Sensor Data
- `orange` (`#f4a261`) — PWM / Logic Gates
- `yellow` (`#eab308`) — Switched Outputs / Cathodes
- `white` (`#e4e4e7`) — Reference Voltages
- `teal` (`#2a9d8f`) — I2C SDA / SPI MOSI
- `purple` (`#a855f7`) — I2C SCL / SPI SCK
- `#RRGGBB` — Any custom hexadecimal color code (e.g. `#ff007f`)

Endpoint Resolution:

1. **Breadboard Holes**: `A15` , `E19` , `J30` , `TOP_POS_10` , `BOT_NEG_5`

2. **Component Terminal Identifiers**:

```
wire BAT1.pos -> TOP_POS_1   red
wire BAT1.neg -> TOP_NEG_1   black
wire RL1.com  -> TOP_POS_10  red
wire RL1.no   -> M1.pos      orange
wire M1.neg   -> TOP_NEG_10  black
```

---

8. Live Runtime Controls (`set`)

The `set` command manipulates component states in real-time during simulation:

```
set  <componentId>.<property>  <value>
```

COMPONENT	TARGET PROPERTY	VALUE TYPE	DESCRIPTION
`pot`	`POT1.wiper`	`0.0` .. `1.0`	Wiper rotation percentage
`sw`	`SW1.closed`	`true` | `false`	Actuate switch contacts
`servo`	`SRV1.angle`	`0` .. `180`	Positional servo angle in degrees
`relay`	`RL1.relay`	`true` | `false`	Manually energize / de-energize coil
`sensor` (sonar)	`US1.distance`	`2` .. `400`	Target obstacle distance in cm
`sensor` (dht22)	`TH1.temp`	`-20` .. `60`	Ambient temperature in °C
`sensor` (dht22)	`TH1.humidity`	`10` .. `95`	Relative humidity percentage
`sensor` (pir)	`PIR1.motion`	`true` | `false`	Motion detection trigger
`sensor` (sound)	`MIC1.sound`	`30` .. `100`	Sound amplitude level in dB
`sensor` (soil)	`SOIL1.moisture`	`0` .. `100`	Soil water saturation level %
`sensor` (hall)	`HALL1.magnet`	`true` | `false`	Magnetic field proximity flag
`encoder`	`ENC1.encoder`	Integer	Quadrature pulse counter step
`keypad`	`KP1.key`	`'0'`..'9', `'A'`..'D'	Simulate keypress on 4×4 matrix
`res`	`R1.value`	Resistance (Ω)	Live resistance adjustment

---

9. Instrumentation & Probing (`probe`)

Connect the digital multimeter's probes to any electrical node:

```
# Attach Red Probe (+)
probe red   TOP_POS_10

# Attach Black Probe (-)
probe black TOP_NEG_10

# Set Multimeter Mode
@probe_mode VOLTS_DC
```

When simulation is active (`@sim on`), the multimeter display immediately reflects the exact MNA-solved electrical potential difference, current loop, or resistance across the chosen points.

---

10. End-to-End Verified Circuit Examples

Example 1: NE555 Astable LED Flasher

```
@circuit "NE555 1Hz Flasher"
@description "Classic astable multivibrator producing square wave flashes"
@author "Ohmic Engine"
@version "2.0"
@board full_830
@temp 25
@sim on

# Power & Passives
bat    BAT1   9V
res    R1     10k       maxP=0.25W
res    R2     47k       maxP=0.25W
res    R_LED  330       maxP=0.25W
cap    C1     10uF      electrolytic
cap    C2     0.01uF    ceramic
ic555  U1
led    LED1   red       Vf=1.9V

# Placements
place BAT1  bench
place U1    E20
place R1    A27-A28
place R2    B27-B26
place C1    E26-E21
place C2    E25-E21
place R_LED E23-E18
place LED1  D18-D17

# Power Rails
wire BAT1.pos -> TOP_POS_1   red
wire BAT1.neg -> TOP_NEG_1   black

# 555 Pin Wiring
wire TOP_POS_1 -> E28        red     # Pin 8 (VCC)
wire TOP_POS_1 -> E24        red     # Pin 4 (RESET)
wire TOP_NEG_1 -> E21        black   # Pin 1 (GND)
wire E22 -> E26              blue    # Pin 2 (TRIG) to Pin 6 (THRES)
wire D17 -> TOP_NEG_1        black   # LED Cathode to GND

# Probing
probe red   E23
probe black E21
@probe_mode FREQUENCY
```

Example 2: Transistor DC Motor Switch with Flyback Diode

```
@circuit "2N2222 Motor Driver Switch"
@description "Logic switch driving high-current DC motor with back-EMF protection"
@author "Ohmic Engine"
@version "2.0"

bat    BAT1   9V
res    R_BASE 1k
sw     SW1    tactile
diode  D_FLY  1N4007
npn    Q1     2N2222    hfe=150
motor  M1     200

place BAT1  bench
place M1    bench
place SW1   E15-F15
place R_BASE A15-A20
place Q1    E20:E21:E22
place D_FLY F21-F22

wire BAT1.pos -> TOP_POS_1   red
wire BAT1.neg -> TOP_NEG_1   black
wire TOP_POS_1 -> B15        red
wire B20      -> B21         orange
wire TOP_NEG_1 -> B20        black
wire M1.pos   -> TOP_POS_1   red
wire M1.neg   -> F22         blue
wire F21      -> TOP_POS_1   red
```

Example 3: Ultrasonic Obstacle Radar with I2C OLED Display

```
@circuit "Sonar Radar Monitor"
@description "HC-SR04 sonar module feeding graphical SSD1306 OLED"
@author "Ohmic Engine"
@version "2.0"

bat     BAT1   9V
reg     REG1   AMS1117_3V3
mcu     MCU1   nano
sensor  US1    hcsr04
disp    OLED1  oled

place BAT1  bench
place MCU1  E10
place US1   B30:B31:B32:B33
place OLED1 B40:B41:B42:B43

wire BAT1.pos -> MCU1.vin    red
wire BAT1.neg -> MCU1.gnd    black
wire MCU1.5v  -> US1.vcc     red
wire MCU1.gnd -> US1.gnd     black
wire MCU1.5v  -> OLED1.vcc   red
wire MCU1.gnd -> OLED1.gnd   black

set US1.distance 35
```

---

11. Compiler Diagnostics & Troubleshooting

ERROR CODE / MESSAGE	PROBABLE CAUSE	FIX / RECOMMENDED SYNTAX
`Unknown component type '<type>'`	Typo in declaration keyword	Check spelling against supported types (`res` , `bat` , `led` , `ic555` , `ic` , `sensor`).
`Component '<id>' has no placement`	Missing `place` statement	Add `place <id> <location>` or `place <id> bench` .
`Invalid hole ID '<id>'`	Out-of-bounds row/column	Ensure column is $1..63$, row is $A..J$, or valid rail ID (`TOP_POS_1..50`).
`Short circuit detected between rails`	Direct jumper from POS to NEG	Review wiring for direct bus tie without series load.
`Component DESTROYED (Thermal Burnout)`	$P = I^2 R > P_{max}$ rating exceeded	Increase current limiting resistance or use higher wattage rating (`maxP=1W`).
`Reverse breakdown / Open circuit`	Diode or LED reversed	Swap anode/cathode pin connections in wiring.

Generated by Ohmic Electronic Workbench Engine • Advanced Agentic CAD System